

Cursor CLI on an Ubuntu VPS

Included-usage assessment for Nullink, StoryCast, and Axis

Prepared 30 September 2026 from official Cursor and xAI documentation. No software was installed, no settings were changed, and no agent task was run.

Cursor CLI can sign in with the same Cursor account as a Pro+ subscription. Official docs say individual plans do not add on-demand charges unless on-demand usage is turned on. Facts below come from pages fetched on 30 September 2026. One conflict with Cursor staff replies on the community forum is marked separately from the docs.

1. Subscription and billing

Verified. Browser login authenticates the CLI as your Cursor account. The authentication page does not name Pro, Pro+, or Ultra, but it says the login prompts you to authenticate with your Cursor account, and credentials are then securely stored locally. A Cursor user API key from Dashboard → API Keys is the other method. The SDK docs say user API keys bill to that user's plan, and that SDK runs use the same pricing, request pools, and Privacy Mode rules as runs from the IDE and Cloud Agents. The CLI auth page does not repeat that sentence. Service accounts, which are a separate non-human login, are Enterprise-only.

Sources: [CLI authentication](#), [SDK](#), [Service accounts](#).

Included usage is two monthly pools on the account, not a CLI-only meter.

- **Cursor Models:** Grok 4.7, Grok 4.6, Grok 4.5, and Composer 2.5.
- **Other Models:** third-party models at that model's API price. Pro, Pro Plus, and Ultra include this pool.

Pro Plus is \$60/month and both pools are listed as Included. The live pricing page does not publish a dollar cap for Pro+. Usage resets with the monthly billing cycle, and unused usage does not roll over. Model choice, Fast mode, and long context change how fast the included pools are consumed. Grok 4.7 Fast is \$4 / \$1 / \$12 per million tokens versus \$2 / \$0.50 / \$6 for standard Grok 4.7. Long context above 256k is 2×, and Fast plus long context is 3×. Those rates draw the included pools first.

New CLI installs default to Auto. All Auto modes bill at the list price of the model each request is routed to, and a third-party route draws Other Models. Subagents can run a named third-party model even when the picker shows Auto, Grok, or Composer, and those requests bill Other Models.

Sources: [Models and pricing](#), [Usage and limits](#), [CLI changelog](#), 6 July 2026.

Shared allowance. The docs describe these pools as plan pools, visible in editor settings and on the usage dashboard. The 13 July 2026 CLI changelog says `/usage` shows your account's included-usage meters. There is no sentence that says the CLI, editor, cloud agents, and Grok Bot share one allowance.

Cloud Agents are a separate billing sentence: they are charged at API pricing for the selected model, and you are asked to set a spend limit when you first start using them. Pro Plus includes access to Cloud Agents. The docs do not say those tokens are free inside the included pools. Background Agents are the old name for Cloud Agents. The CLI can hand a chat to a Cloud Agent by prefixing a message with `&`.

Grok Bot has its own weekly included grant. See section 2.

Hard stop. On an individual plan, turn on-demand usage off in spending settings. The overages page says that stops requests once included usage runs out. The editor then shows a notification, and you can enable on-demand

or upgrade. There is no documented CLI flag or environment variable for this.

A spend limit is only a cap on on-demand, and on-demand usage must be enabled to view and set spend limits. Setting the limit to “No Limit” removes the cap. The official spend-limit page does not document a \$0 limit. Spend-limit enforcement is not instant, so usage can briefly exceed the limit. Cursor then applies a temporary spend-limit credit and bills you up to the current limit. Raising the limit later in the same cycle can make that credit billable. Leave the limit unchanged, or turn off on-demand usage, to keep the credit. Spend alerts do not stop usage.

Sources: [Usage-based charges](#), [Spend limits](#).

Setting	Where	Effect when you want included usage only
On-demand usage, off	cursor.com/dashboard → Spending	Documented stop after included usage is gone
Monthly spend limit	Same Spending tab, only after on-demand is enabled	Caps on-demand. It is not the off switch
“No Limit”	Same control	Removes the cap
On-demand monthly limit	Grok Bot → Settings, same web Monthly Limit	Caps Grok Bot overflow. A running Bot can finish past it
Provider API keys	Cursor Settings → Models	Your provider bills you. On individual plans those requests do not draw Cursor pools
<code>CURSOR_API_KEY</code>	Dashboard → API Keys, then the shell	Cursor user key. SDK says it bills that user’s plan. It is not a BYOK provider key

Charges that can still happen with on-demand off

- Included-pool consumption is covered by the \$60 subscription until the pools are exhausted. Auto and subagents can empty Other Models without a separate invoice.
- Bring-your-own-key requests are billed by OpenAI, Anthropic, Google, Azure, or Bedrock.
- Cloud Agent handoff (&), Cloud Agent API runs, and the first-run Cloud Agent spend limit are not documented as blocked by the individual on-demand toggle.
- Bugbot uses included usage first, then pauses until the next cycle if on-demand is off.
- MCP calls still spend model tokens. A third-party MCP server can bill you at that provider.
- The Auto-review classifier runs on a small Cursor-managed model. Today that is Claude 4.5 Haiku or GPT-5.4 Mini. Whether those calls draw your pools is not documented.
- The Cursor Token Rate of \$0.25 per million tokens is documented for Teams and Enterprise, not for individual Pro+.
- Max Mode is legacy request-based pricing only. Current usage-based plans do not include it. `/max-mode` toggles Max Mode on legacy request-based plans.

Checking remaining usage. The Spending tab shows real-time usage for both pools, remaining allowance, on-demand charges, and the reset date. Billing & Invoices splits Included Usage and On-Demand Usage.

`agent status` and `agent about` report authentication, endpoint, version, system, and account info. They support `--format text|json`. They are not documented as remaining-balance commands. `agent models` lists models for the account. No individual-plan usage REST endpoint is documented. Admin and Analytics usage APIs are Enterprise.

`/usage` is in the 13 July 2026 changelog and is absent from the current slash-command reference. Treat it as changelog-documented and unverified against the current command list. It is an interactive slash command, not a scriptable JSON balance API. SDK `Agent.getUsage()` returns cost for that agent's runs, and `chargedCents` is 0 for plan-included usage. It is not a remaining-balance API.

2. Grok Bot and SuperGrok

Verified from Cursor's Grok Bot docs. Grok Bot does not have a separate subscription. Access is included with Cursor Pro, Pro+, Ultra, or a self-serve Teams seat. You can also link an individual SuperGrok, SuperGrok Plus, SuperGrok Heavy, or X Premium+ subscription. Pro+ includes generous weekly usage, below Ultra. SuperGrok Lite, Team, and Enterprise cannot link.

Grok Bot signs in with the Cursor account. Its usage is metered on that Cursor account, not in the Grok app. Weekly usage resets weekly. If it runs out and on-demand is enabled, extra usage is billed through Cursor and counts toward the same Monthly Limit as the web Spending tab. If on-demand is off, Grok Bot stops and says you have reached the Grok Bot usage limit.

A SuperGrok link is a usage grant, not a Cursor plan. Linking SuperGrok on top of a Cursor plan does not add usage. The docs explicitly say that on Pro+, linking SuperGrok Plus does not add Grok Bot usage. The two subscriptions can both stay active and keep billing separately. Buying or canceling one does not cancel the other. The link itself is permanent. You cannot unlink it yourself.

Sources: [Grok Bot plans](#), [Grok Bot FAQs](#), [Link SuperGrok](#).

Does Morc depend on Pro+, SuperGrok, or both? With an active Pro+ account, Cursor's docs say Grok Bot access comes from Pro+. A linked SuperGrok subscription does not add a second allowance. Morc's routines spend that weekly grant. An hourly routine, or a Slack listener on a busy channel, can use a week of usage in a day. A saved routine that never fires does not spend usage. A test run does. Bots, memory, and routines live on the Cursor account and its cloud computer. The docs do not say that canceling SuperGrok deletes them.

Canceling SuperGrok while Pro+ stays active. The documented model is that Pro+ already includes Grok Bot, the link does not stack, and SuperGrok billing is with xAI, not Cursor. The grant from the link applies while SuperGrok or X Premium+ stays active. Because that grant adds nothing on top of Pro+, the docs support Morc continuing on the Pro+ weekly grant. They do not contain a page that says canceling SuperGrok has no effect on bots, memory, routines, or tools. Check the meter title before you cancel. If Weekly usage is titled with a SuperGrok tier rather than Pro+, contact Cursor support with both account emails before canceling.

Using Cursor from Grok Bot. Cursor's help pages do not spell out a double charge. They do say on-demand is one shared cap. Cloud Agents, which Grok Bot can start, are charged at API pricing. Cursor staff on the forum, which is not the docs site, say the Bot's own chats, routines, and its own computer use the weekly Grok Bot pool, while a Cloud Agent the Bot launches bills the normal Cursor plan, and Bugbot on those pull requests uses Cursor included usage. That forum explanation is the clearest operational account, and it is not written on the docs pages.

Conflicting official text.

- [xAI's Grok Bot FAQ](#) says: if you have both a Cursor and a SuperGrok subscription, Grok Bot uses whichever has more usage.
- [xAI's expansion post](#) says Grok Bot comes with its own usage, separate from your Grok and Cursor plans, so anything you hand off to a Bot will not count against your existing usage.
- Cursor's help center says the plans do not stack and that on-demand overflow is billed through Cursor.

Those three cannot all be true for the same account. Cursor's help center is the operational billing document for a Cursor login. The xAI sentences disagree with it. This needs an account check, not an assumption.

What would be needed to verify this, without passwords or tokens. Cursor account email, whether the plan shown is individual Pro+, the Spending tab's on-demand state and both pool percentages, the reset date, the Grok Bot Usage & Billing title on the weekly meter, whether a Grok account is linked and which SuperGrok tier is still renewing, whether Cloud Agent spend is enabled, whether any provider keys are saved under Settings → Models, and whether Bugbot is enabled on the repositories More touches.

3. Installation and authentication

Supported install, from the docs. Linux is supported. The documented command is:

```
curl https://cursor.com/install -fsS | bash
agent --version
```

Add `~/local/bin` to `PATH`. The CLI auto-updates. `agent update` updates manually. The 11 August 2026 changelog says the install script falls back to `wget` or `python3` if `curl` is broken. No Node version or glibc version is documented for the CLI binary. Behind a firewall, allow `*.cursor.sh` and `*.cursorapi.com`.

Android login. `NO_OPEN_BROWSER=1 agent login` prints a URL you can open manually. The 6 July 2026 changelog says that during interactive `agent login` you can press `q` for a QR code of that same URL and scan it from a phone. Narrow terminals and non-interactive sessions show the URL only. The docs do not say Android by name. A phone browser that can open the printed URL is the documented path. The Android note on the Cloud Agent page is for `cursor.com/agents` as a progressive web app, not for `agent login`.

Credentials. The auth page says they are stored locally and that `agent logout` clears them. The changelog says `~/cursor` stores CLI credentials, and `AGENT_CLI_CREDENTIAL_STORE=file` stores them unencrypted in an owner-only file. The filename, file mode, and refresh interval are not documented. Dashboard API keys can be revoked by deleting the key. Browser-login revocation, beyond `agent logout` and whatever the dashboard does to sessions, is not documented. Expired tokens prompt re-login.

Dedicated user. The install lands in the invoking user's home. Nothing in the install page requires root. The Linux sandbox AppArmor package does: `sudo dpkg -i of cursor-sandbox-apparmor_0.6.0_all.deb`. A dedicated non-root user matches the documented per-user install. The docs never use the phrase "dedicated non-root user."

Unattended subscription auth. Headless mode is `agent -p`. The docs tell scripts to export `CURSOR_API_KEY`. That key is a Cursor user key and, per the SDK, bills the user's plan. It does not switch the CLI onto OpenAI or Anthropic billing. Browser login for the same Unix user is the other unattended path, because the credentials stay on disk. Either path still consumes included usage. Neither path is documented as a way around the on-demand switch. Do not set a provider key, and do not start a Cloud Agent, if paid overflow is forbidden.

`--force` and `--yolo` allow file changes and command execution without confirmation. The headless page says that without `--force`, changes are only proposed. The parameters page says `--print` has access to all tools, including write and shell. Those two pages disagree. Treat `--force` as the switch that applies edits.

4. Automation and Nullink

Documented support. The CLI help page says yes: use headless mode for scripts, CI, and GitHub Actions. The Terms grant a limited right to use the Service and incorporate an Acceptable Use Policy. No ban on personal scripting appears in the CLI docs. Subscriptions are sold only through cursor.com. Reseller or shared-account use is the documented suspension risk, not a worker you run for your own repositories.

What a worker can rely on.

- Noninteractive: `-p / --print`, `--mode ask` or `--mode plan`, `--model`.
- JSON: `--output-format json` returns one object with `result`, `session_id`, and optional `request_id`. On failure, the process exits non-zero, writes to stderr, and emits no JSON object.
- Progress: `--output-format stream-json` emits newline-delimited JSON. The `system / init` event includes `model`, `cwd`, `session_id`, and `apiKeySource` (`env`, `flag`, or `login`). `tool_call` events have `started` and `completed`.
- Resume: `--resume [chatId]`, `--continue`, `agent resume`, `agent ls`, `agent create-chat`.
- Exit codes: the code-review sample checks `$?`. Specific numeric codes are not documented.
- Cancellation is not reliable for child processes. The 11 August 2026 changelog says headless runs wait for delegated subagents instead of cutting off while background shells or dev servers are still running. An earlier changelog entry says stopping a turn preserves in-flight shell output and moves running tools to the background instead of killing them. The 6 July changelog says quitting no longer waits for MCP servers or background tasks to wind down. A worker that needs a hard stop has to kill the process group itself, and the docs do not promise that child processes die with the CLI.

Nullink. The CLI does not document an inbound “launch this task” API. A worker starts it as a subprocess, captures stdout and stderr, and uses `session_id` to resume. `agent worker start` is a different product: a private Cloud Agent worker with `/healthz`, `/readyz`, and `/metrics`. It connects the machine to Cursor’s cloud control plane. That is not required for a local Nullink supervisor, and it is a Cloud Agent billing path.

MCP means the CLI calls MCP servers. It does not publish itself as an MCP server. It reads project `.cursor/mcp.json` and `~/.cursor/mcp.json`, the same servers as the editor. `agent mcp list`, `list-tools`, `login`, `enable`, and `disable` manage those servers. `Mcp(server:tool)` permissions gate them. `agent acp` is Agent Client Protocol over stdio, not MCP.

Pointing this CLI at the same Nullink gateway that can itself start Cursor or Codex tasks can recurse. Keep that gateway out of the CLI’s `mcp.json` unless the exposed tools are read-only and cannot launch another agent.

Limits without a paid fallback. Before a task, read the Spending tab. There is no documented scriptable `remaining-balance` call for an individual plan. Pin `--model` to a Cursor Models model such as Composer 2.5 or Grok 4.7. Do not use Auto if you need to stay out of Other Models. Pass `--mode ask` for read-only work. Do not pass `&`, do not set `CURSOR_API_KEY` to a provider key, and do not enable on-demand in the worker. If stderr reports an authentication failure, stop and require an interactive `agent login`. If it reports a usage limit, stop until the reset date. Do not retry on another model, another key, or a Cloud Agent. A non-zero exit with no JSON result is a failed task, not a signal to fall back to a paid API.

5. Safe repository access

Worktrees are an edit location, not a security boundary. `--worktree [name]` creates a checkout under `~/.cursor/worktrees/`. `--workspace` sets the repository root. The using page says `--worktree` only

changes where the agent makes file edits inside that project. `--add-dir` adds more directories. Shell commands still run on the host unless the sandbox stops them.

Control	What it does	Enforced how
<code>permissions.deny</code> in <code>~/.cursor/cli-config.json</code> or the project <code>.cursor/cli.json</code>	Blocks Shell, Read, Write, WebFetch, and Mcp patterns. Deny wins over allow	CLI policy. Shell matching is the first token, with optional <code>command:args</code> globs. <code>bash -c</code> is not the same token as <code>systemctl</code>
<code>approvalMode</code> : allowlist, auto-review, unrestricted	Who is asked before a tool runs	Application. Auto-review can make mistakes. Run Everything and <code>--force</code> / <code>--yolo</code> skip prompts except explicit denies
Linux sandbox	Landlock filesystem limits and seccomp. Network blocked by default, then opened by <code>sandbox.json</code>	Kernel, only if the kernel is 6.2 or newer with Landlock v3 and unprivileged user namespaces. Otherwise the CLI falls back to approval prompts. The CLI package does not ship the AppArmor profile. Ubuntu needs the documented <code>cursor-sandbox-apparmor</code> package. Some commands bypass the sandbox and ask for approval
<code>.cursorignore</code> and default ignores of <code>.env</code> and <code>.git/</code>	Hides files from Agent file tools	Not a sandbox. Terminal commands and MCP tools run outside Cursor's file access controls
Browser, file-deletion, and external-file protections	Extra approval even in automatic modes	Documented as protections that can still ask. Not described as a jail
OS user, directory mode, and absence of <code>docker.sock</code> , <code>systemd</code> , and secret files	The real production boundary	Unix permissions. The CLI docs do not provide this for you

Run Modes are best-effort guardrails rather than a hard security boundary. Cloud Agents do not use Run Modes at all. They run in their own VM and do not ask for approval.

Production secrets and deploys. Run the CLI as a user that cannot read production env files, SSH keys, or the Docker socket, and that cannot restart services. Deny `Shell(sudo)`, `Shell(systemctl)`, `Shell(docker)`, `Shell(kubectl)`, and `Read(.env*)` in the CLI config, and also keep those files unreadable by that user. Do not use unrestricted approval mode on this host. Do not approve MCP servers that can deploy or restart. `--trust` skips the workspace trust prompt. It does not grant a sandbox escape.

Cursor and Codex. Nothing in the Cursor docs coordinates with Codex. Use separate Git worktrees and separate Unix users. Cursor's `--worktree` does not lock Codex out of the main checkout. Give each task one worktree path, and do not pass the other agent's worktree to `--workspace` or `--add-dir`. Nulllink should refuse to start a second writer when a lock for that worktree is held.

6. Testing, auditing, and resources

Builds and tests. The CLI has shell, file, search, and web-fetch tools. A build or test runs if the shell allowlist permits it. That uses your machine's CPU, RAM, and disk. No CLI CPU, RAM, disk, or concurrency number is published.

Browser checks. Web fetch is documented. A full browser on a headless VPS is not documented for the standalone CLI. `agent worker start --computer-use` needs a Linux X11 display such as `:0`. Cloud Agents have their own desktop and browser, and they are the API-priced path. A headless Playwright run is just a shell command you allow. It is not a documented Cursor browser product.

What you can capture.

- `stream-json`: model name, working directory, tool calls, assistant text, `session_id`, `request_id`, durations.
- Headless transcripts are written. The changelog describes JSONL transcripts. A stable directory and a secret-redaction switch for those transcripts are not documented. Plugin config forms mask fields named like tokens. That is not general log redaction.
- `/logs` shows the debug log path. It copies the path to the clipboard, which a headless session does not have.
- Diffs are the Git diff in the worktree. The interactive review UI is Ctrl+R. A script should run `git diff` itself.
- Test results are whatever the test command prints into the tool-call event.
- Your own task id is not a documented field. Put it in the prompt and in the wrapper's log line. `--header` adds a request header. The docs do not say that header is stored as a correlation id.
- Redaction belongs in the wrapper before logs are stored. Tool results can contain file contents.

Cancellation. Do not assume child processes die. The changelog says the opposite for in-flight shells and for headless runs that wait on background shells. A supervisor that must stop work should track the process group and the worktree, then decide explicitly what to kill. Expect leftover build processes until you do that.

Capacity to plan for, as an operating limit rather than a Cursor quota. One local agent can saturate a compile. Two agents on one VPS contend for CPU and for the same ports. Grok Bot's cloud computer is separate and shared by every Bot on the account. Keep the production app's cgroup untouched, and put the CLI user in its own systemd slice with a CPU, memory, and process cap you choose. Cursor does not document those numbers.

A. Recommended setup for this VPS

Keep production processes and the agent user apart. Create a non-root user that can read only the repositories you intend to edit, with no Docker socket, no systemd control, and no production secret directory.

Leave on-demand usage off on the Cursor dashboard before the first login. Confirm both pool percentages and the reset date on the Spending tab, and confirm Grok Bot's weekly meter and its on-demand toggle. Remove any provider keys under Settings → Models. Leave Cloud Agent spend unset, and do not use `&` or `agent worker start` from this host.

Install the CLI only into that user's home, with `~/local/bin` on that user's `PATH`. Log in once with `NO_OPEN_BROWSER=1 agent login` and finish the URL on your phone. After that, unattended runs use the stored subscription login. Do not export a provider API key. A Cursor user API key is optional and still bills this same plan.

Set `approvalMode` to allowlist, enable the sandbox, install the AppArmor package if user namespaces are restricted, and deny deploy and secret paths. Run each task with `--workspace` pointed at a fresh Git worktree, `--model` pinned to Composer 2.5 or Grok 4.7, and `--mode ask` until you intentionally allow edits. Nullink starts `agent -p --output-format stream-json`, records `session_id` and your task id, and does not retry on a usage-limit error.

Codex keeps its own worktree. Nulllink holds one writer lock per worktree.

B. Verified billing safeguards and unresolved risks

Safeguards the docs actually state

- Individual on-demand usage is off until you enable it.
- With it off, requests stop when included usage runs out, and Bugbot pauses.
- Grok Bot stops at the weekly grant when on-demand is off.
- The SuperGrok link does not add usage on top of Pro+.
- A Cursor user API key bills the user's plan. It is not a hidden second meter in the SDK docs.
- Bring-your-own-key is the path that bills another provider. Do not add those keys.

Unresolved risks

- Cloud Agents are charged at API pricing, and the docs do not say the individual on-demand toggle blocks them. Grok Bot can launch them. Forum staff say that launch bills the Cursor plan.
- Auto and subagents can spend the Other Models pool while the picker still shows Grok or Composer.
- Spend limits are not a hard stop in the middle of a run, and they require on-demand to be on. A \$0 limit is forum advice, not a documented setting.
- xAI's FAQ and news post disagree with Cursor's help center about whether SuperGrok and Cursor usage combine.
- The Auto-review classifier's billing is not documented.
- There is no documented individual-plan API that returns remaining allowance, so a worker cannot prove headroom before a task.
- `/usage` is in the changelog and missing from the current slash-command page.

C. Minimal installation and read-only verification plan

Do this only after the dashboard check. Do not send a prompt. Any `agent -p` or interactive task can consume included usage.

1. On the dashboard, record that on-demand is off, both pools' remaining allowance, the reset date, Grok Bot's weekly title, and that no provider keys are saved.
2. Create the dedicated user and a directory it can write. Do not add it to docker or sudo.
3. As that user, run the curl install and `agent --version`.
4. Run `NO_OPEN_BROWSER=1 agent login`, open the URL on the phone, then `agent status --format json` and `agent about --format json`. Confirm the account email. Stop there.
5. Optionally run `agent models` and confirm Composer 2.5 or Grok 4.7 is listed. Do not start a chat.
6. As root, only if you want the sandbox later, install the documented AppArmor package and confirm the kernel has Landlock. That package install is the only root step.
7. Write the deny rules before the first real task. The first real task stays out of scope until you explicitly allow it.

D. Account-specific questions still open

- Is this login an individual Pro+ subscription, or a Teams seat where on-demand is on by default?
- Is on-demand currently off, and what are the two pool percentages and the reset date?
- What title does Grok Bot show on Weekly usage: Pro+, or a SuperGrok tier?
- Is a SuperGrok subscription linked, and is it still renewing?
- Are Cloud Agent spend, Bugbot, and any Settings → Models provider keys enabled?
- Does Morc start Cloud Agents today, or does it only use the Bot's own computer?
- Which repositories have uncommitted work that a worktree must not touch?

Documentation fetched 30 September 2026. This file records that assessment. It does not install Cursor CLI, change account settings, or start an agent.